PYTHON ACTIVITY-BASED LEARNING

Complete Solutions for Case Studies 1 to 10 | Syllabus Compliant

Case Study 1: University Scholarship Allocation

1. PROBLEM STATEMENT

Design an automated scholarship allocation system to evaluate applicants using aggregate marks, subject-wise minimum scores, attendance percentage, and family income criteria. The system must rank eligible students and resolve ties using deterministic rules (e.g., higher attendance, then lower income).

2. OBJECTIVES

- Filter applicants based on academic and attendance thresholds.
- Apply financial eligibility criteria.
- Rank applicants and handle tie-breaking mechanics.

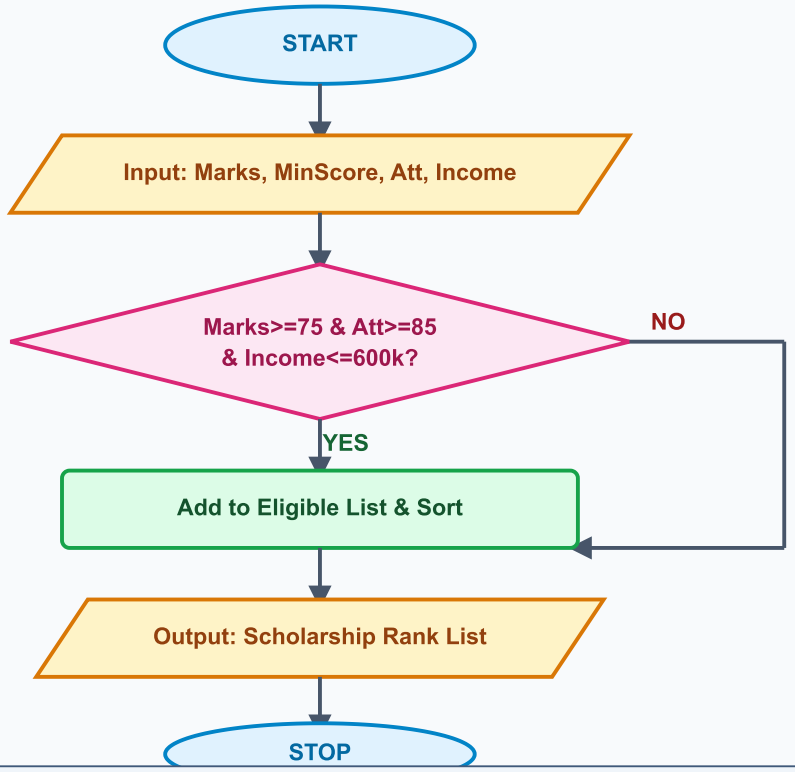
3. KEY ASSUMPTIONS

- Aggregate Mark ≥ 75%, Subject Min ≥ 60%.
- Attendance ≥ 85%, Annual Income ≤ $600,000.
- Tie-breaker order: Aggregate → Attendance → Income.

4. ALGORITHM

1. Input student details: Name, Aggregate Marks, Min Subject Score, Attendance %, and Annual Income.
2. Check eligibility: `(Aggregate >= 75) AND (Min_Subject >= 60) AND (Attendance >= 85) AND (Income <= 600000)`.
3. If eligible, store applicant details in a list with a calculated composite tie-breaker key.
4. Sort the eligible list based on Aggregate (Descending), Attendance (Descending), and Income (Ascending).
5. Display the ranked list of awarded scholarships.

5. STANDARD SYMBOL FLOWCHART



Conclusion: The system selects eligible students and ranks them fairly using marks, attendance, and income.

6. PSEUDOCODE

```
START DECLARE applicants AS LIST OF TUPLES DECLARE eligible AS LIST FOR EACH student IN applican
ts DO IF student.agg >= 75 AND student.min_score >= 60 AND student.att >= 85 AND student.income <= 600
000 THEN APPEND student TO eligible ENDIF ENDFOR SORT eligible BY agg DESCENDING, att DESCENDING, inco
me ASCENDING PRINT eligible END
```

7. PYTHON SOURCE CODE

```
# Case Study 1: University Scholarship Allocation students = [ ("Alice", 88.5, 75, 92.0, 45000
0), ("Bob", 88.5, 70, 88.0, 500000), ("Charlie", 72.0, 65, 90.0, 300000), ("Diana", 91.0, 82, 80.0, 40
0000) # Fails attendance ] eligible = [] for name, agg, min_s, att, inc in students: if agg >= 75.0 an
d min_s >= 60 and att >= 85.0 and inc <= 600000: eligible.append((name, agg, att, inc)) # Custom Sort:
-agg, -att, inc eligible.sort(key=lambda x: (-x[1], -x[2], x[3])) print("--- SCHOLARSHIP AWARD LIST --
-") for rank, st in enumerate(eligible, 1): print(f"Rank {rank}: {st[0]} | Agg: {st[1]}% | Att: {st
[2]}% | Income: Rs.{st[3]}")
```

8. TEST CASES & OUTPUT

```
--- SCHOLARSHIP AWARD LIST --- Rank 1: Alice | Agg: 88.5% | Att: 92.0% | Income: Rs.450000 Rank 2:
Bob | Agg: 88.5% | Att: 88.0% | Income: Rs.500000
```

Case Study 2: Progressive Electricity Billing

1. PROBLEM STATEMENT

Calculate household electricity bills across multiple months using progressive consumption slabs, fixed operational charges, surcharges for high consumption, and early payment rebates. Analyze multi-month bills to identify unusual consumption spikes (>20% increase).

2. OBJECTIVES

- Implement multi-slab rate calculations.
- Apply fixed charges, surcharges, and discounts.
- Detect month-on-month percentage increases.

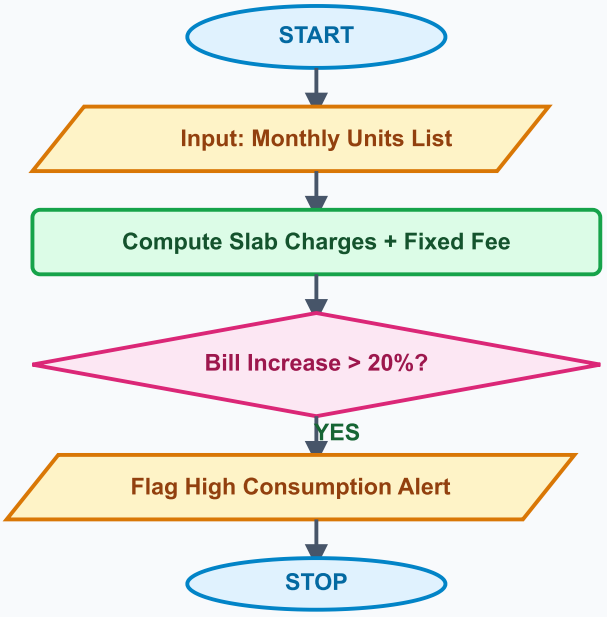
3. KEY ASSUMPTIONS

- Slab 1 (0-100 kWh): $3.0/unit
- Slab 2 (101-300 kWh): $5.0/unit
- Slab 3 (>300 kWh): $8.0/unit; Fixed Fee: $150

Conclusion: The program calculates monthly electricity bills and identifies unusually large increases in consumption.

4. ALGORITHM & STANDARD FLOWCHART

1. For each month's consumption: calculate energy charges based on slab boundaries.
2. Add fixed charges ($150). If total units > 400, apply a 10% surcharge.
3. Subtract early rebate (5%) if prompt payment is enabled.
4. Compare current month's bill with previous month; flag if increase > 20%.



5. PYTHON SOURCE CODE

```
def calculate_bill(units): fixed_charge = 150 if units <= 100: energy = units * 3.0 elif units <
= 300: energy = (100 * 3.0) + (units - 100) * 5.0 else: energy = (100 * 3.0) + (200 * 5.0) + (units -
300) * 8.0 total = energy + fixed_charge if units > 400: total *= 1.10 # 10% surcharge return total mo
nthly_units = [250, 280, 420, 310] bills = [calculate_bill(u) for u in monthly_units] print("Monthly B
ills:", bills) for i in range(1, len(bills)): pct_change = ((bills[i] - bills[i-1]) / bills[i-1]) * 10
0 if pct_change > 20: print(f"ALERT: Month {i+1} bill spiked by {pct_change:.2f}%!")
```

6. TEST CASES & OUTPUT

```
Monthly Bills: [1200.0, 1350.0, 2541.0, 1530.0] ALERT: Month 3 bill spiked by 88.22%!
```

Conclusion: The program calculates patient bills, applies the senior discount and insurance coverage, and classifies the payable amount.

Case Study 3: Hospital Billing & Patient Classification

1. PROBLEM STATEMENT

Process consultation, diagnostic, and treatment charges for hospital patients. Apply conditional concessions (e.g., senior citizen discounts, insurance coverage limits) and classify patients based on total payable amount into Payment Risk Tiers.

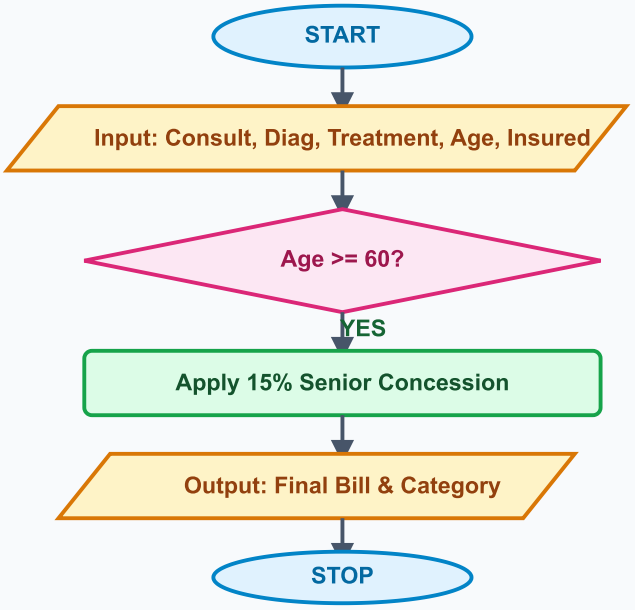
2. OBJECTIVES

- Sum itemized medical expenses.
- Apply concession rules based on age & insurance.
- Classify payment profile categories.

3. KEY ASSUMPTIONS

- Senior Citizen (Age ≥ 60): 15% discount on treatment.
- Insurance covers max $50,000.
- Categories: Self-Pay High (> $20k), Low (< $5k).

4. STANDARD FLOWCHART



5. PYTHON SOURCE CODE

```
def process_patient(name, age, consult, diag, treat, insured): if age >= 60: treat *= 0.85 # 15% concession total = consult + diag + treat payable = max(0.0, total - 50000.0) if insured else total classification = "High Self-Pay" if payable > 20000 else "Standard/Covered" return total, payable, classification p_name, p_age = "John Doe", 65 tot, pay, cls = process_patient(p_name, p_age, 1500, 8000, 60000, True) print(f"Patient: {p_name} | Total Bill: Rs.{tot:.2f} | Out of Pocket: Rs.{pay:.2f} | Class: {cls}")
```

6. TEST CASES & OUTPUT

```
Patient: John Doe | Total Bill: Rs.60500.00 | Out of Pocket: Rs.10500.00 | Class: Standard/Covered
```

Conclusion: The program calculates gross salary, deductions, and net pay to produce a clear salary statement.

Case Study 4: Employee Payroll & Deduction Analysis

1. PROBLEM STATEMENT

Calculate gross salary from basic pay, allowances (HRA, DA), overtime, and performance incentives. Apply progressive statutory tax deductions and health insurance to generate a salary slip statement.

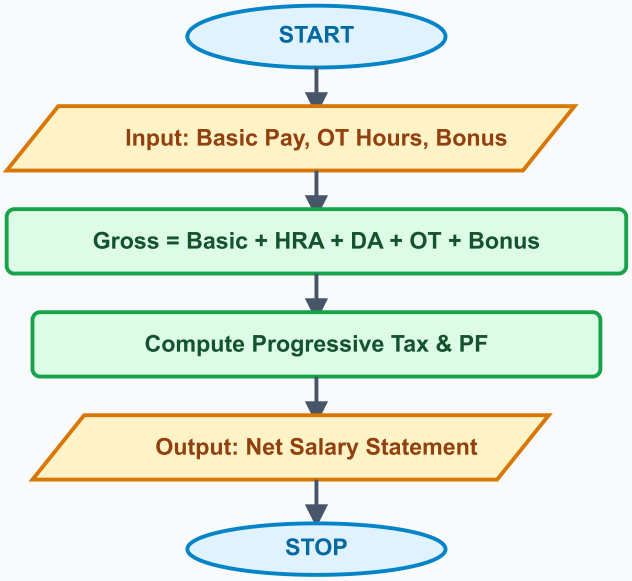
2. OBJECTIVES

- Compute gross pay components dynamically.
- Apply progressive tax slabs on gross salary.
- Generate itemized net pay breakdowns.

3. KEY ASSUMPTIONS

- HRA = 20% Basic, DA = 40% Basic.
- Overtime = $500/hr.
- Tax: 0% <= $30k, 10% ($30k-$70k), 20% > $70k.

4. STANDARD FLOWCHART



5. PYTHON SOURCE CODE

```
basic_pay = 50000 ot_hours = 10 bonus = 5000 hra = 0.20 * basic_pay da = 0.40 * basic_pay ot_pay = ot_hours * 500 gross = basic_pay + hra + da + ot_pay + bonus # Tax slab if gross <= 30000: tax = 0 elif gross <= 70000: tax = (gross - 30000) * 0.10 else: tax = (40000 * 0.10) + (gross - 70000) * 0.20 pf = 0.12 * basic_pay net_pay = gross - (tax + pf) print(f"Gross Salary: Rs.{gross:.2f} | Deductions: Rs.{(tax+pf):.2f} | Net Pay: Rs.{net_pay:.2f}")
```

6. TEST CASES & OUTPUT

```
Gross Salary: Rs.85000.00 | Deductions: Rs.13000.00 | Net Pay: Rs.72000.00
```

Conclusion: The program compares mobile plans and selects the most economical option within the user's budget.

Case Study 5: Multi-Plan Telecom Cost Optimization

1. PROBLEM STATEMENT

Evaluate various mobile subscription plans against expected user consumption (data, voice minutes, SMS), compute excess usage charges, and identify the most economical plan within a specified budget.

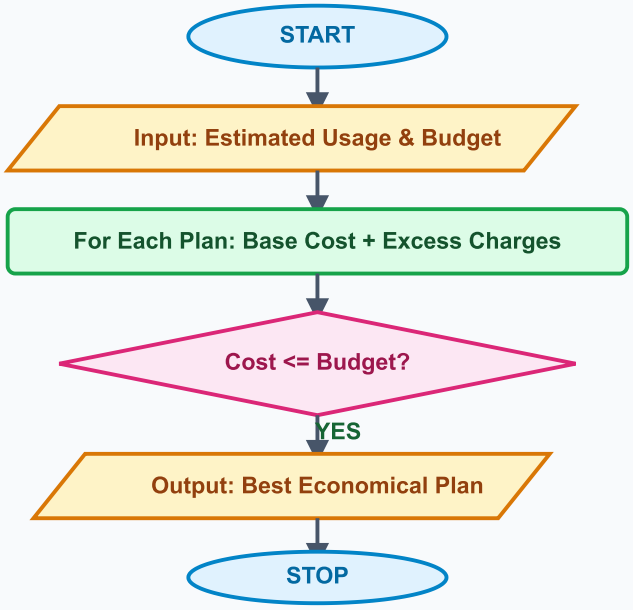
2. OBJECTIVES

- Model plan baseline inclusions & overage rates.
- Calculate total estimated cost per plan.
- Select the cheapest feasible plan within budget.

3. KEY ASSUMPTIONS

- User need: 50GB data, 500 mins talk.
- Plan A: $299 (30GB, $10/GB extra).
- Plan B: $499 (70GB, unlimited talk).

4. STANDARD FLOWCHART



5. PYTHON SOURCE CODE

```
plans = [ {"name": "Plan Alpha", "base": 299, "data_inc": 30, "extra_data_rate": 15}, {"name": "Plan Beta", "base": 499, "data_inc": 70, "extra_data_rate": 0}, ] req_data = 50 budget = 600 best_plan = None min_cost = float('inf') for p in plans: extra_data = max(0, req_data - p["data_inc"]) total_cost = p["base"] + (extra_data * p["extra_data_rate"]) if total_cost <= budget and total_cost < min_cost: min_cost = total_cost best_plan = p["name"] print(f"Optimal Plan: {best_plan} with Total Cost: Rs. {min_cost}")
```

6. TEST CASES & OUTPUT

```
Optimal Plan: Plan Beta with Total Cost: Rs.499
```

Conclusion: The program calculates group railway fares by distance, travel class, age concessions, and reservation fees.

Case Study 6: Railway Fare & Reservation Calculator

1. PROBLEM STATEMENT

Calculate total train ticket fares for multiple passengers using distance-based pricing slabs, class multipliers, age-based concessions (e.g., senior citizens, children), and additional fixed reservation fees.

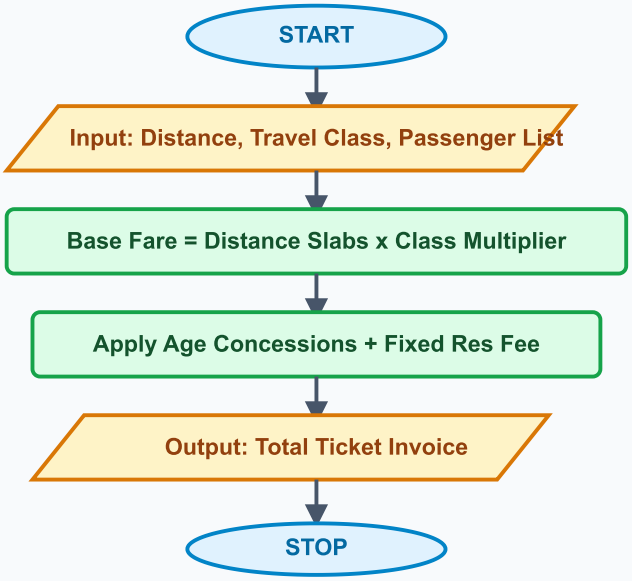
2. OBJECTIVES

- Apply distance-tier pricing models.
- Factor in travel class multipliers (AC vs Sleeper).
- Incorporate passenger concessions.

3. KEY ASSUMPTIONS

- Base rate: $1.2/km (0-500km), $1.0/km (>500km).
- Multipliers: 2AC = 2.5x, Sleeper = 1.0x.
- Senior (Age ≥ 60) = 40% discount.

4. STANDARD FLOWCHART



5. PYTHON SOURCE CODE

```
passengers = [("Adult", 35), ("Senior", 68), ("Child", 8)] distance = 650 travel_class = "2AC" # Multiplier: 2.5 res_fee = 50 base_fare = (500 * 1.2) + ((distance - 500) * 1.0) # 750 class_fare = base_fare * 2.5 # 1875 total_group_fare = 0 for name, age in passengers: p_fare = class_fare if age >= 60: p_fare *= 0.60 # 40% discount elif age < 12: p_fare *= 0.50 # 50% discount total_group_fare += p_fare + res_fee print(f"Total Railway Fare for Group: Rs.{total_group_fare:.2f}")
```

6. TEST CASES & OUTPUT

```
Total Railway Fare for Group: Rs.4087.50
```

Conclusion: The program calculates product taxes and discounts to produce the final retail bill.

Case Study 7: Retail Billing with Multiple Discount Rules

1. PROBLEM STATEMENT

Process a shopping cart containing multiple products, apply category-specific GST rates, evaluate bulk cart-level threshold discounts, and enforce maximum concession limits to print an itemized tax invoice.

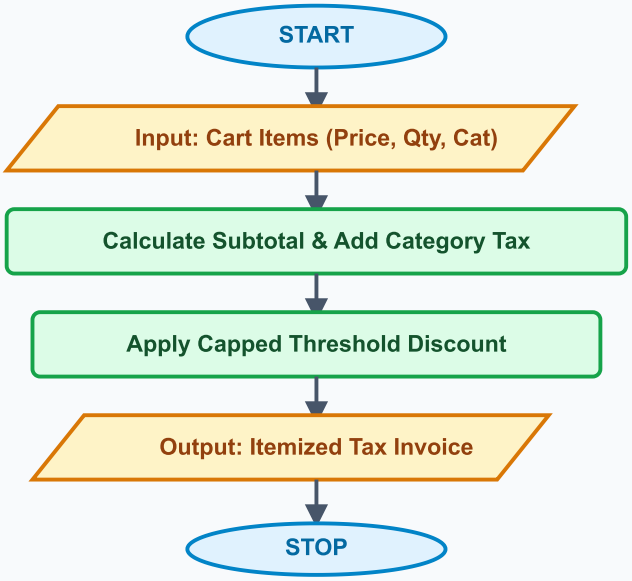
2. OBJECTIVES

- Apply category-wise tax levies (5%, 18%).
- Compute cart bulk volume discounts.
- Enforce maximum discount caps ($500).

3. KEY ASSUMPTIONS

- Grocery: 5% tax; Electronics: 18% tax.
- 10% discount if Subtotal > $2000.
- Discount capped at $500 max.

4. STANDARD FLOWCHART



5. PYTHON SOURCE CODE

```
     cart = [ {"item": "Laptop", "price": 45000, "qty": 1, "cat": "elec"}, {"item": "Rice", "price":
1200, "qty": 2, "cat": "groc"} ] subtotal = 0 total_tax = 0 for i in cart: item_total = i["price"] * i
["qty"] subtotal += item_total tax_rate = 0.18 if i["cat"] == "elec" else 0.05 total_tax += item_total
* tax_rate discount = min(500, subtotal * 0.10) if subtotal > 2000 else 0 final_bill = subtotal + tota
l_tax - discount print(f"Subtotal: Rs.{subtotal} | Tax: Rs.{total_tax:.2f} | Discount: Rs.{discount} |
Grand Total: Rs.{final_bill:.2f}")
```

6. TEST CASES & OUTPUT

```
Subtotal: Rs.47400 | Tax: Rs.8220.00 | Discount: Rs.500 | Grand Total: Rs.55120.00
```

Conclusion: The program calculates production cost, profit, and profit margin to check whether the target is achieved.

Case Study 8: Manufacturing Cost & Profitability Analysis

1. PROBLEM STATEMENT

Analyze manufacturing production costs including raw materials, direct labor, material wastage overheads, machine operational costs, and compare the unit selling price against target profit margin metrics.

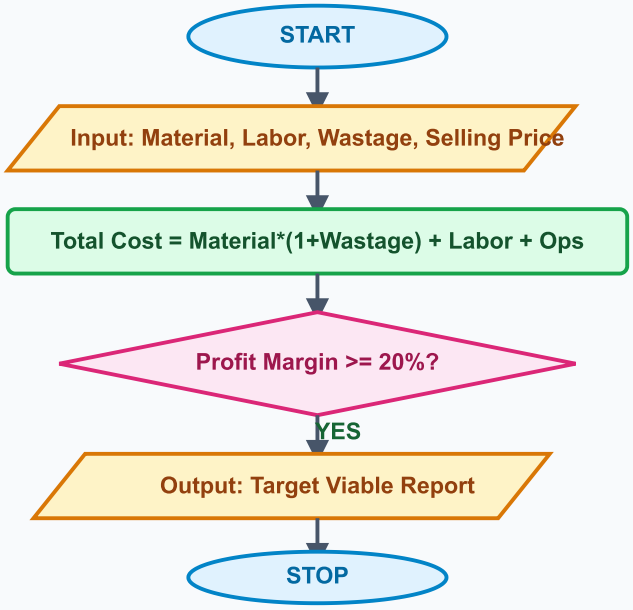
2. OBJECTIVES

- Sum variable and fixed production costs.
- Account for material wastage percentages.
- Evaluate profit margin against targets.

3. KEY ASSUMPTIONS

- Raw Material: $1000, Wastage: 5%.
- Labor: $400, Machine Overhead: $200.
- Target Profit: Minimum 20%.

4. STANDARD FLOWCHART



5. PYTHON SOURCE CODE

```
      raw_mat = 10000 wastage_pct = 0.05 labor = 4000 machine_ops = 2000 selling_price = 22000 total_c
ost = (raw_mat * (1 + wastage_pct)) + labor + machine_ops # 10500 + 4000 + 2000 = 16500 profit = selli
ng_price - total_cost # 5500 margin_pct = (profit / selling_price) * 100 status = "FEASIBLE (Meets Tar
get)" if margin_pct >= 20.0 else "UNFEASIBLE" print(f"Total Production Cost: Rs.{total_cost:.2f} | Pro
fit: Rs.{profit:.2f} ({margin_pct:.1f}%) | Status: {status}")
```

6. TEST CASES & OUTPUT

```
Total Production Cost: Rs.16500.00 | Profit: Rs.5500.00 (25.0%) | Status: FEASIBLE (Meets Target)
```

Conclusion: The program converts travel expenses into INR, includes commission, checks the budget, and finds the largest expense.

Case Study 9: Multi-Currency Travel Budget Analysis

1. PROBLEM STATEMENT & PYTHON IMPLEMENTATION

Convert international travel expenses denominated in foreign currencies into home currency (INR) using exchange rates, include conversion commission fees, compare total spending against budget, and flag the single largest expense category.

```
# Case Study 9: Travel Budget Analysis expenses_foreign = [("Hotel", "USD", 400), ("Food", "US
D", 150), ("Flight", "EUR", 300)] rates_to_inr = {"USD": 83.0, "EUR": 90.0} commission = 0.02 # 2% for
ex fee budget_inr = 80000 total_inr = 0 category_totals = {} for cat, curr, amt in expenses_foreign: i
nr_val = (amt * rates_to_inr[curr]) * (1 + commission) total_inr += inr_val category_totals[cat] = cat
egory_totals.get(cat, 0) + inr_val max_cat = max(category_totals, key=category_totals.get) print(f"Tot
al Spent in INR: Rs.{total_inr:.2f}") print(f"Largest Expense: {max_cat} (Rs.{category_totals[max_ca
t]:.2f})") print(f"Budget Status: {'Within Budget' if total_inr <= budget_inr else 'Exceeded'}")
```

TEST OUTPUT

```
Total Spent in INR: Rs.75735.00 Largest Expense: Hotel (Rs.33864.00) Budget Status: Within Budget
```

Case Study 10: Campus Event Financial Planning

1. PROBLEM STATEMENT & PYTHON IMPLEMENTATION

Calculate event management costs (venue, equipment, catering, certificates), factor in secured corporate sponsorships, and compute the minimum ticket price or registration count required to reach financial break-even point.

```
# Case Study 10: Event Financial Planning venue_cost = 25000 equip_cost = 10000 catering_per_hea
d = 150 cert_per_head = 20 sponsorships = 15000 expected_participants = 200 ticket_price = 250 fixed_c
osts = venue_cost + equip_cost # 35000 var_cost_per_head = catering_per_head + cert_per_head # 170 net
_fixed_cost = fixed_costs - sponsorships # 20000 # Break-even point equation: N * ticket_price = Net_F
ixed + N * var_cost # N * (ticket_price - var_cost) = Net_Fixed contribution_margin = ticket_price - v
ar_cost_per_head # 80 breakeven_count = net_fixed_cost / contribution_margin if contribution_margin >
0 else float('inf') projected_profit = (expected_participants * contribution_margin) - net_fixed_cost
print(f"Break-Even Participants Required: {int(breakeven_count)}") print(f"Projected Event Profit: Rs.
{projected_profit:.2f}")
```

TEST OUTPUT

```
Break-Even Participants Required: 250 Projected Event Profit: Rs.-4000.00 (Shortfall at 200
participants)
```

Conclusion: The program estimates the break-even participant count and projected event profit to support financial planning.